Character Device Driver

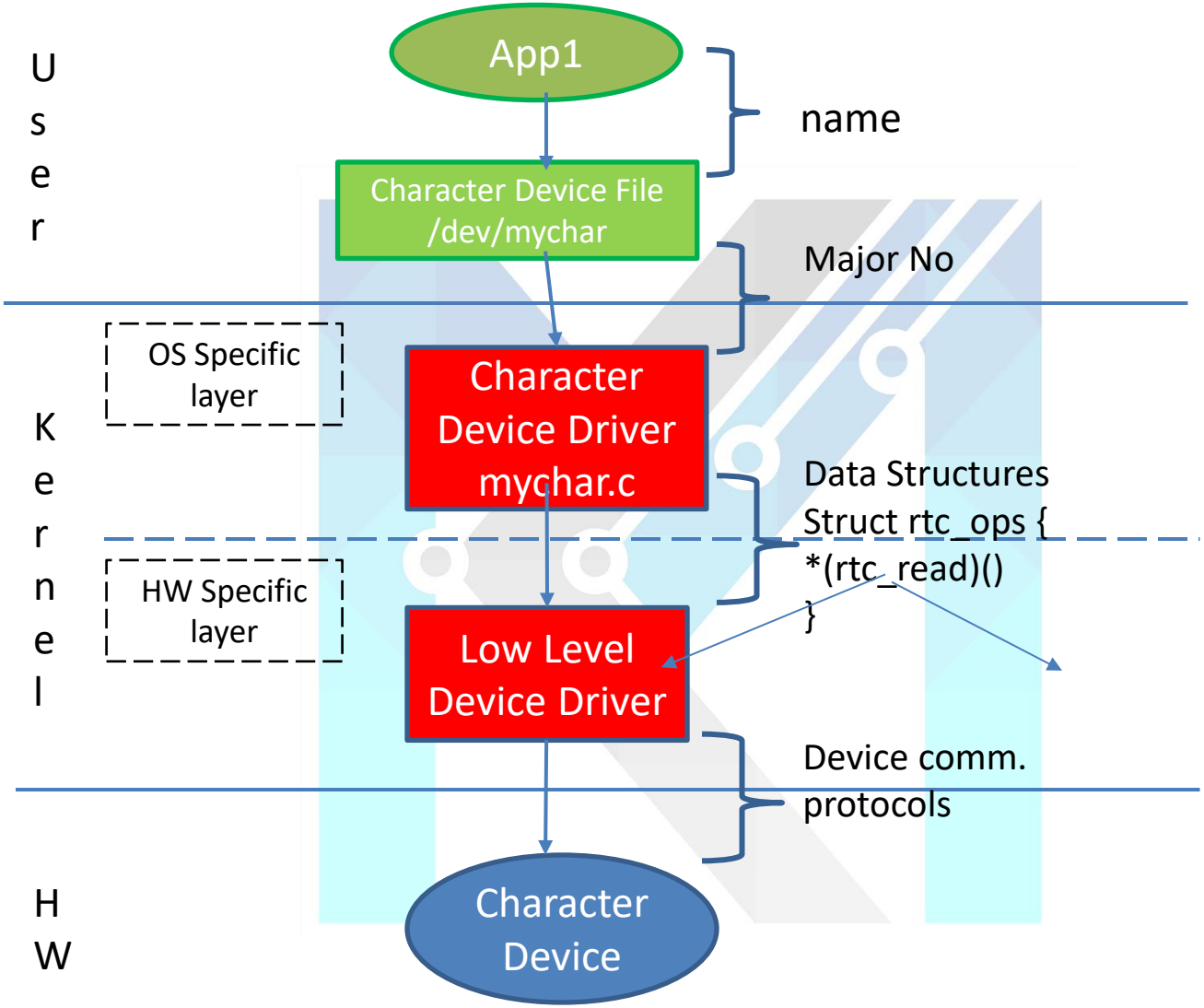
character oriented device

KERNEL MASTERS

Character Device Drivers

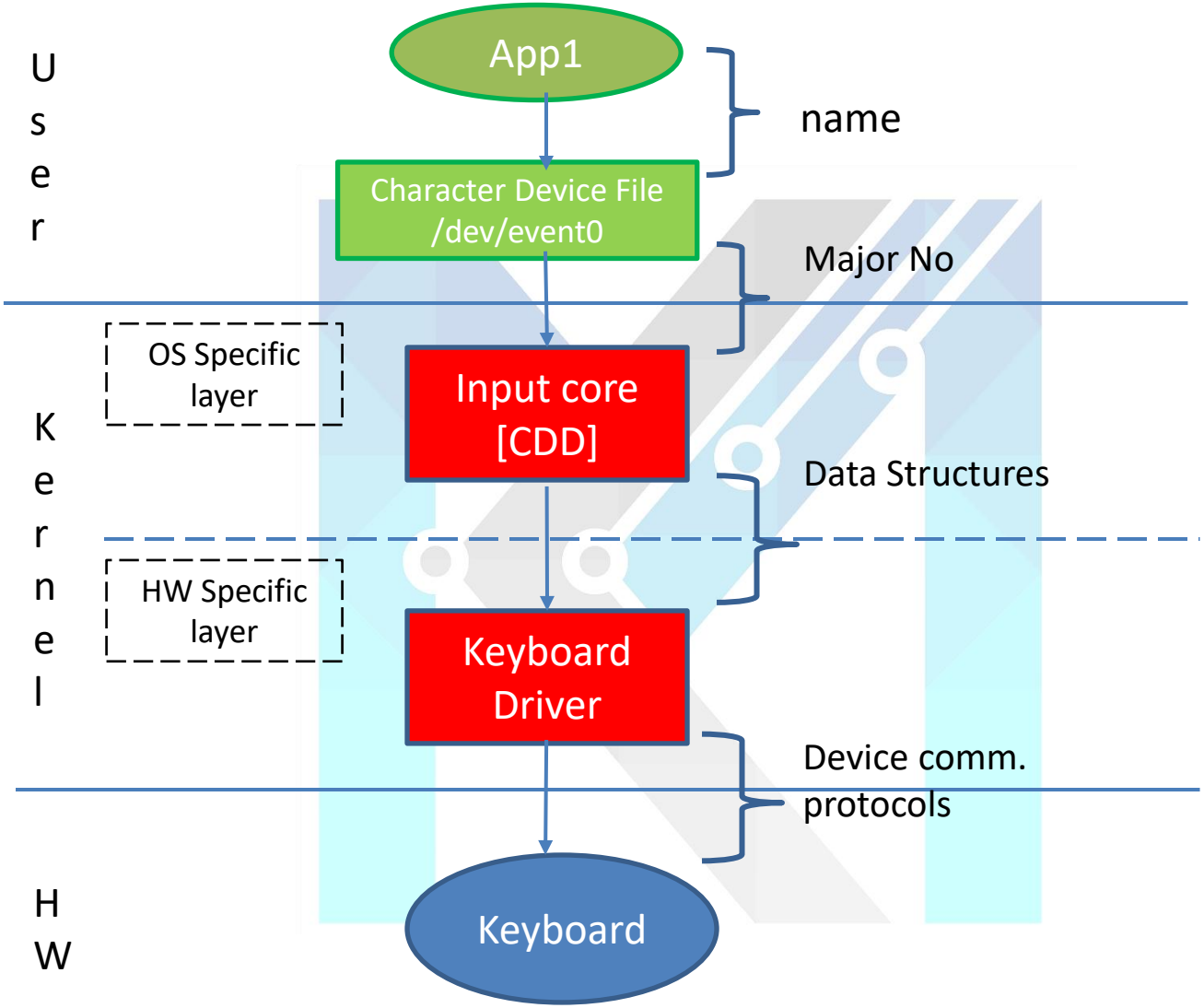
- Character devices can access data sequential fashion where as block devices can access data random fashion.

Character Device Drivers Framework



KERNEL MASTERS

Input Device Drivers Framework



KERNEL MASTERS

CDD Implementation

Step 1: Character Device Driver Registration for major no (or) minor no.

Ex: register_chrdev()

Step 2: Linking the Device File Operations to the device driver operations.

Step 3: Create a character device file using mknod.

Device Files are created by the mknod command:

mknod <name_device> <device_type> <major> <minor> [-m modes]

Example:

mknod /dev/MyCharDevice c 253 15 -m 0666

Step 4: Write an application that initate operation of the driver. (Optional)

Event	User mode	Kernel mode
Load	insmod	mychar_init()
Open	Open()	mychar_open)
Read	Read()	mychar_read()
Close	Close()	mychar_close()
Unload	Rmmod	mychar_exit()

CDD Registration

Static CDD Registration:

static inline int register_chrdev (**0**, const char *name, const struct file_operations *fops)
{
    return __register_chrdev(major, 0, 256, name, fops);
}

Dynamic CDD Registration: (Kernel allocate major no.)

extern int alloc_chrdev_region(dev_t *, unsigned, unsigned, const char *);
Ex: alloc_chrdev_region(&mychar, 0, 2, “mychar”);
254 0
255 1
alloc_chrdev_region(&mychar, 64, 3, “mychar”);
254 64
254 65
254 66

CDD Registration

Static CDD implementation

CDF creation: (one time)

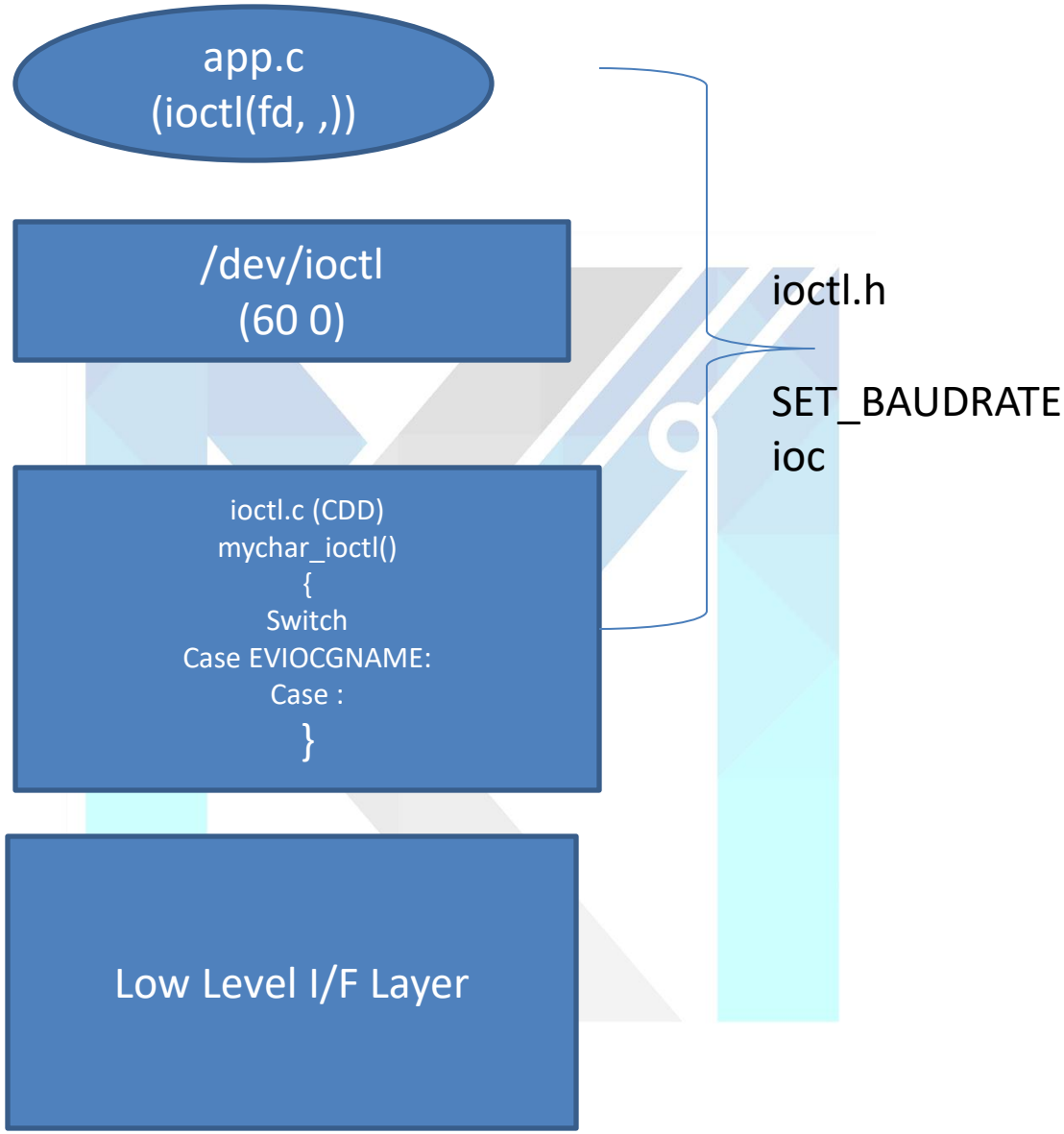
1. Sudo mknod /dev/mychar 60 0
2. Sudo chmod 666 /dev/mychar

1.Edit & build

2. Sudo insmod mychar.ko
3. Cat /dev/mychar
4. Sudo rmmod mychar.ko

Dynamic CDD implementation

1. Edit & build
2. sudo insmod CharDevice.ko
3. cat /proc/devices
4. Sudo mknod /dev/mydev <> 0
5. Sudo chmod 666 /dev/mydev
6. Cat /dev/mydev
7. sudo rmmod mychar.ko

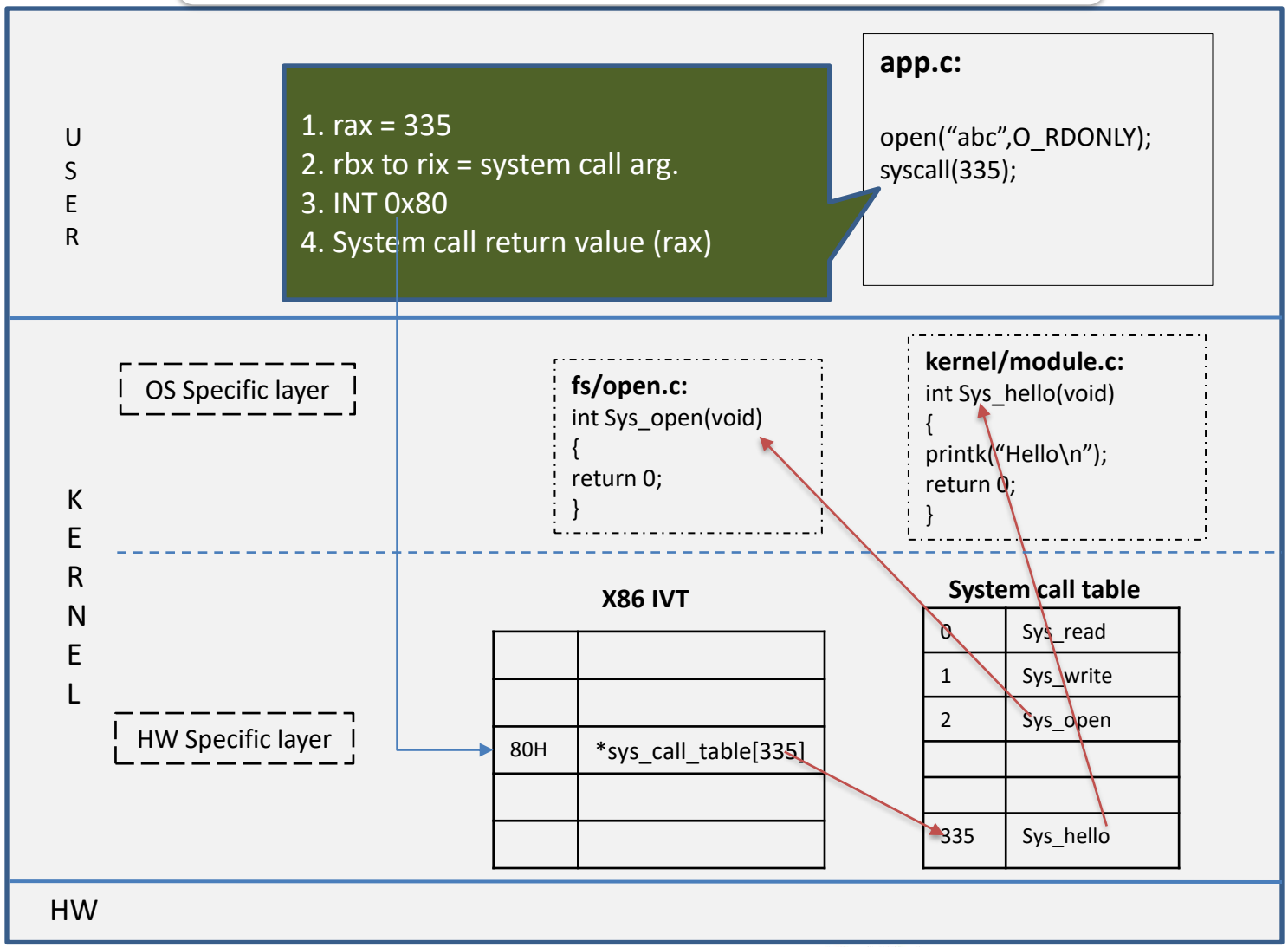


- ioctl() command divide to 4 bit fields:

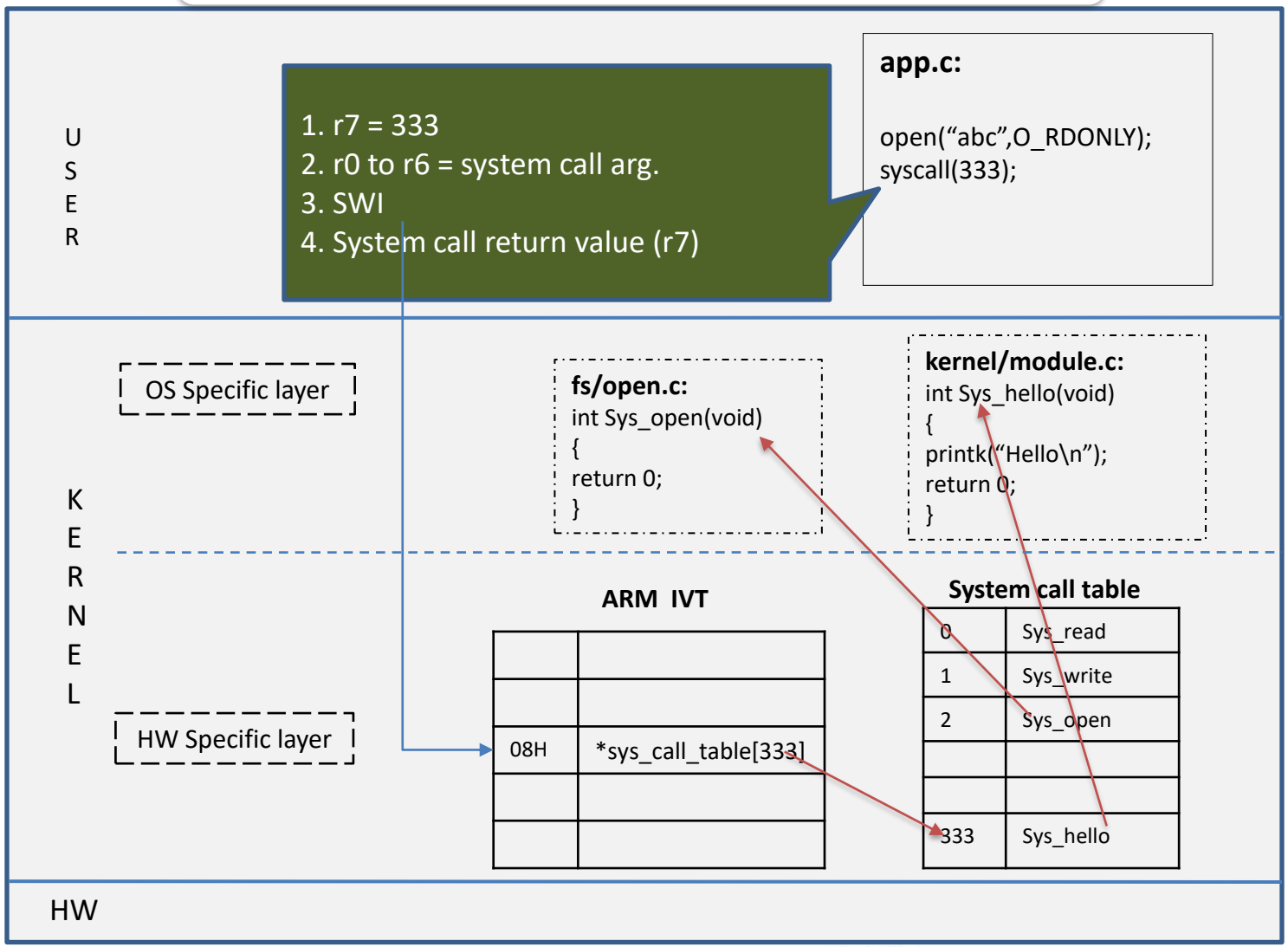
1. Magic Number (8bits - 256) – Device Specific Parameters (Documentation/ioctl/ioctl-number.txt)
2. Sequential number (8bits – 256 - range)
3. read, write and both (2 bits – 4 comm)
4. Size (32-18=14 bits)

1. rax = System call no
2. rbx to rix = system call arg.
3. INT 0x80 (SWI)
4. System call return value (rax)

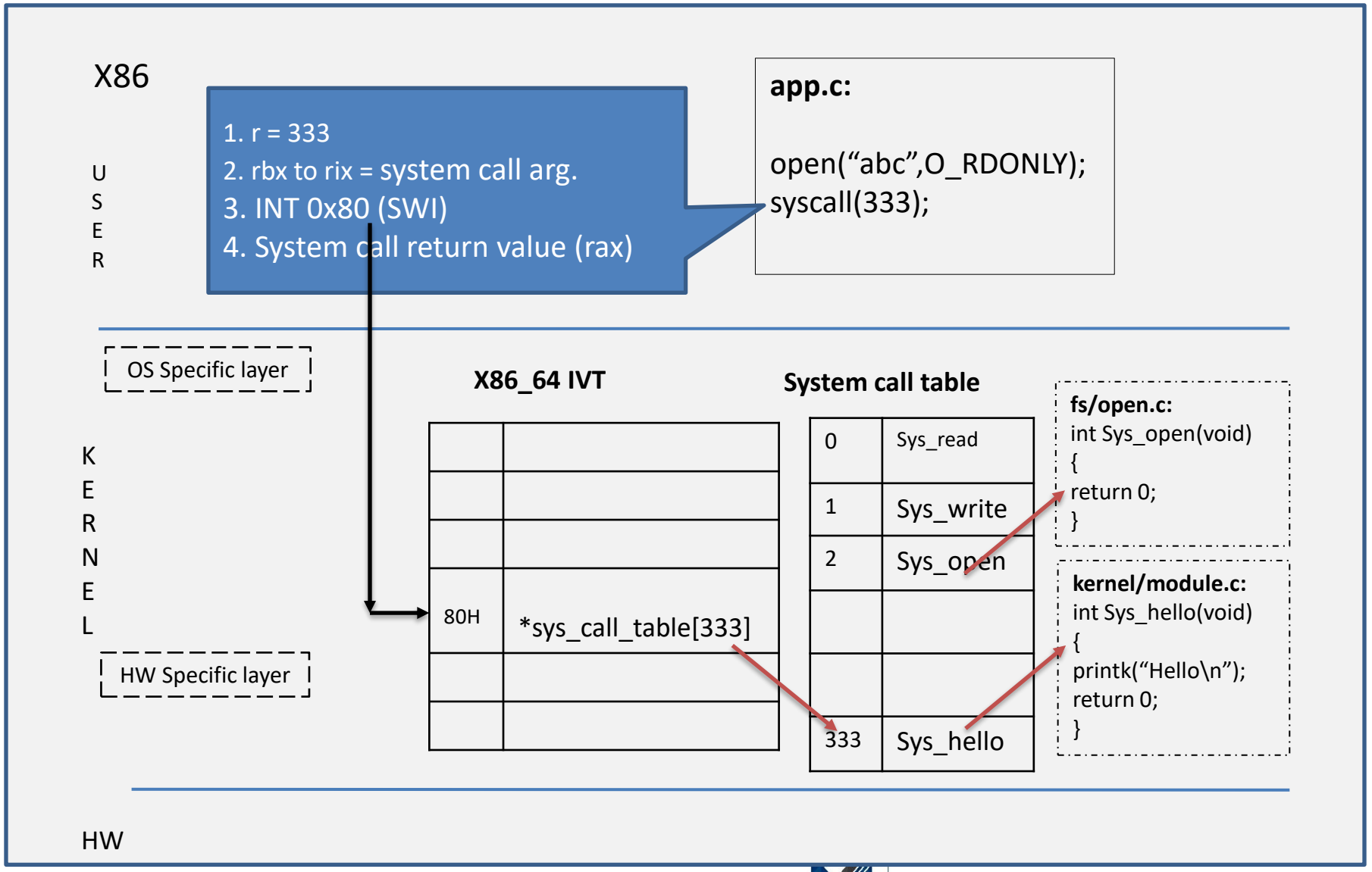
How System Call Works – X86 Architecture

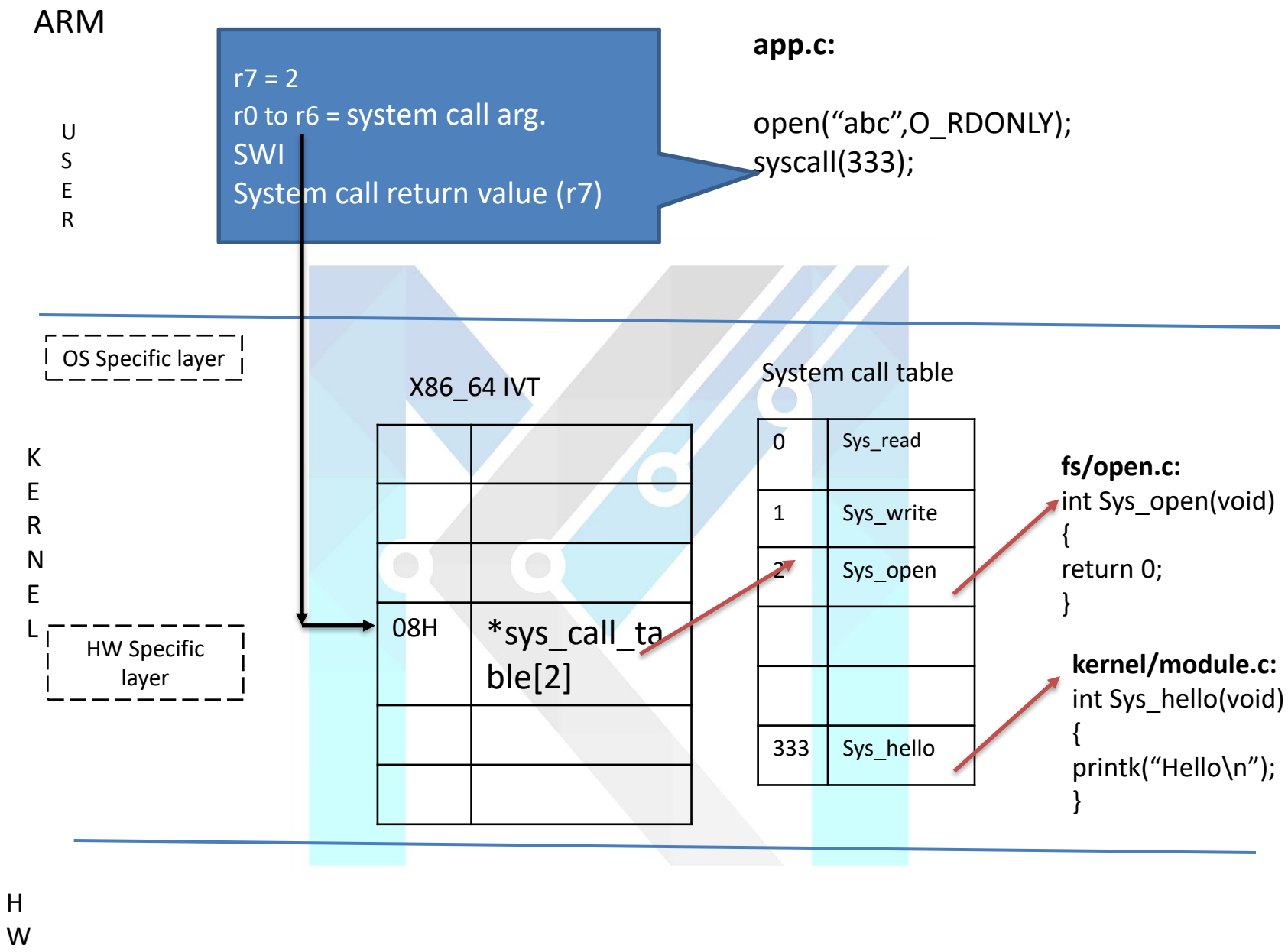


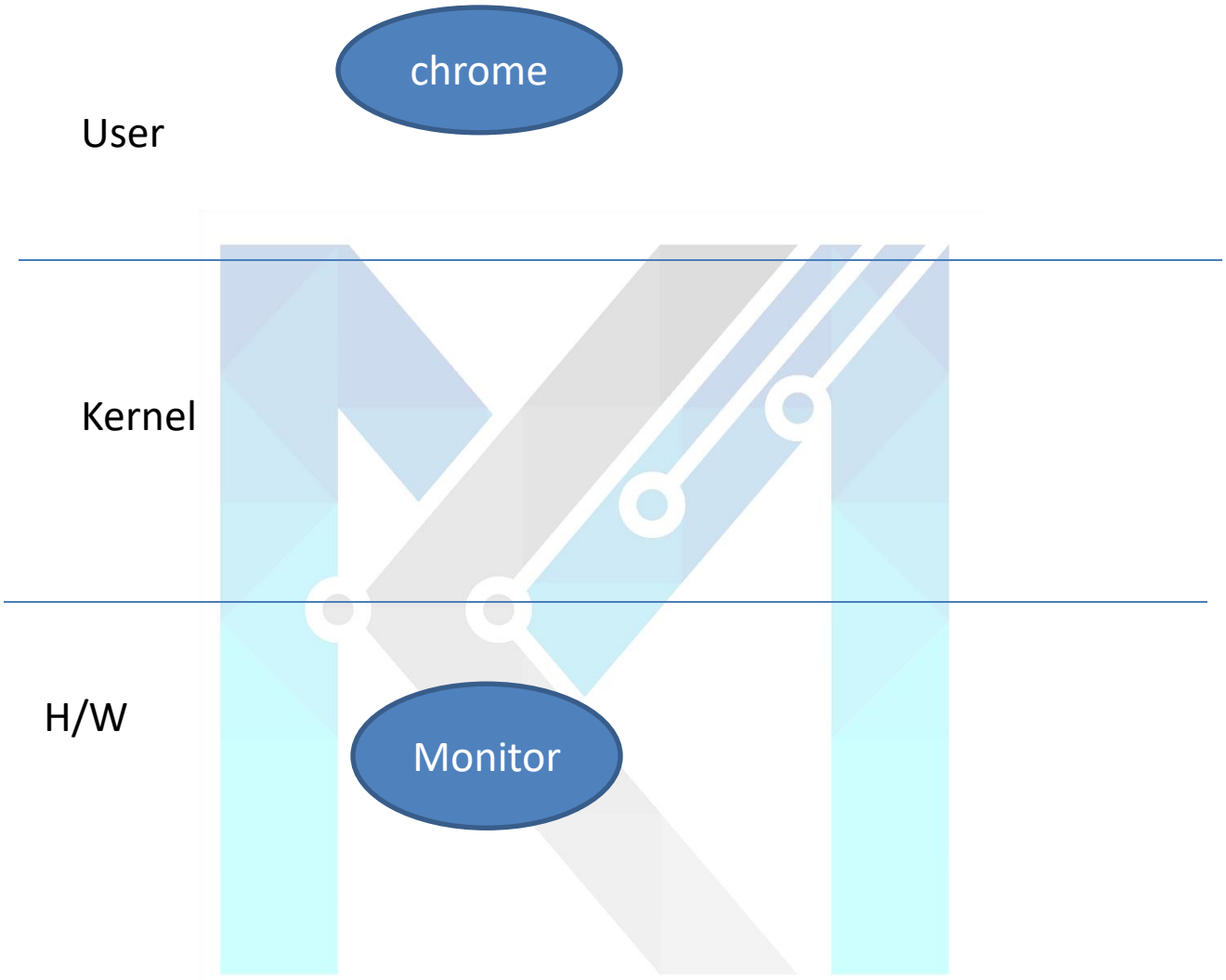
How System Call Works – ARM Architecture



How System Call Works – x86 Architecture







Kernel Initialization steps

Init/main.c: start_kernel()

arch/x86/kernel/traps.c: trap_init()

---

asmlinkage const sys_call_ptr_t
sys_call_table[__NR_syscall_max+1] = {

arch/x86/entry/syscall_64.c : invoke system call table

arch/x86/include/generated/asm/syscalls_64.h

- Interrupt latency
- Spinlock example
- Delay below 1msec
- Shared interrupt line (last arg)
- Update latest kernel compilation issues

Static CDD	Dynamic CDD
Development Cycle 1: 1. Write a driver and Programmer also identify major no. (60) and assign no and build code 2. $ sudo insmod mychar.ko 3. $ sudo mknod /dev/mychar c 60 0 4. Run application program and test driver $ cat /dev/mychar 5. $ sudo rmmod mychar.ko	**Development Cycle 1:** 1. Write a driver and Programmer not assign a major no. and build code. 2. $ sudo insmod mychar.ko 3. $ cat /proc/devices 4. $ sudo mknod /dev/mychar c <ma> 0 5. Run application program and test driver $ cat /dev/mychar 6. $ sudo rmmod mychar.ko
Development Cycle 2: 5. Edit driver program & Rebuild code 6. $ sudo insmod mychar.ko 7. Run application program and test driver $ cat /dev/mychar 8. $ sudo rmmod mychar.ko	**Development Cycle 2:** 5. Edit driver program & Rebuild code 6. $ sudo insmod mychar.ko 7. $ cat /proc/devices 8. $ sudo rm /dev/mychar 9. $ sudo mknod /dev/mychar c <ma> 0 10. Run application program and test driver $ cat /dev/mychar 11. $ sudo rmmod mychar.ko
Development Cycle 3: 5. Edit driver program & Rebuild code 6. $ sudo insmod mychar.ko 7. Run application program and test driver $ cat /dev/mychar 8. $ sudo rmmod mychar.ko	**Development Cycle 3:** 5. Edit driver program & Rebuild code 6. $ sudo insmod mychar.ko 7. $ cat /proc/devices 8. $ sudo rm /dev/mychar 9. $ sudo mknod /dev/mychar c <ma> 0 10. Run application program and test driver $ cat /dev/mychar 11. $ sudo rmmod mychar.ko